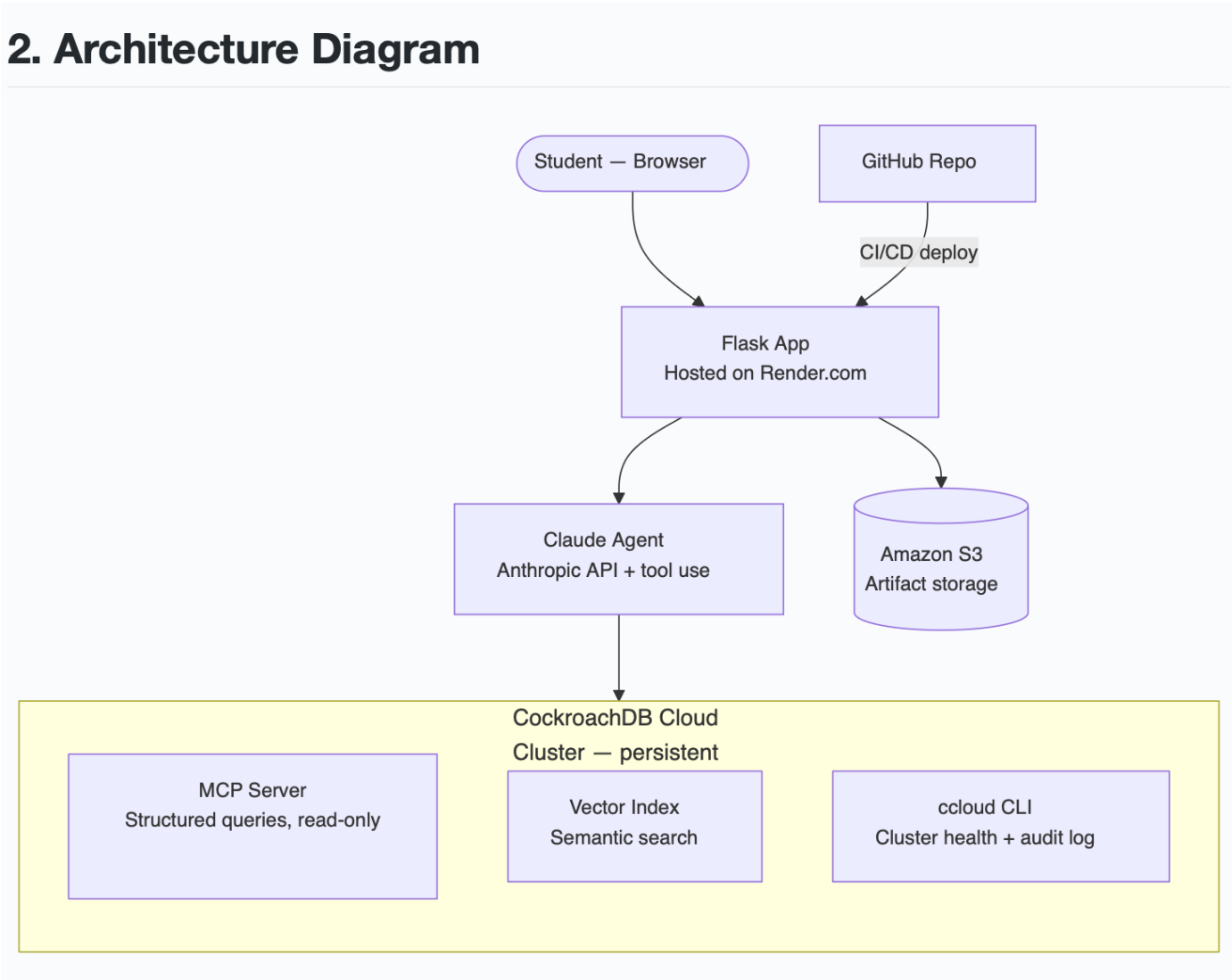
AI Course Advisor — Architecture

An agentic course-recommendation platform that uses CockroachDB as its persistent memory layer to help students choose courses based on their interests, target career, or GPA goals.

1. Overview

A student describes what they want (an interest, a target job, a GPA to hit). A Claude agent reasons over structured university data and semantic course/career matches — both stored in a single CockroachDB Cloud cluster — and returns a ranked, explained set of course recommendations. The cluster is the system's memory: it holds conversation history, student context, in-progress task state, course/career embeddings, and all transactional enrollment data.

2. Architecture Diagram



3. Components

Component	Role
Student (browser)	Submits a goal — interest, target job, or GPA target — through a simple form.
Flask app (Render.com)	Request handling, session management, calls the Claude agent, writes artifacts to S3.
Claude agent (Anthropic API)	Orchestrates tool calls, reasons over structured + semantic results, produces the final recommendation with a short "why this helps you" explanation per course.
CockroachDB Cloud cluster	The persistent memory layer — one cluster, three access paths (MCP, vector index, ccloud CLI).
Amazon S3	Stores generated recommendation reports and dataset backups — the AWS service requirement.
GitHub → Render	CI/CD: push to `main` triggers a Render deploy.

4. Memory layer mapping

This is the part judges will look at closest — memory isn't one thing here, it's five, all living in CockroachDB.

Memory type	Where it lives	How the agent uses it
Conversation history	`conversations` table (`session_id` , `turn` , `role` , `content` , `created_at`)	Agent reads prior turns via MCP so a returning student gets continuity instead of starting cold.
User context	`students` + `student_goals` tables (interests, target job, target GPA, completed courses)	Read via MCP at the start of every session to personalize the query.
Task state	`recommendation_sessions` table (`status` , `last_tool_used` , `partial_results` , `updated_at`)	Written mid-run so a request survives a Render cold-start or retry without losing progress — this is the "memory that never goes down" story in practice.
Embeddings	`VECTOR` columns on `courses` and `career_outcomes`	Queried through the distributed vector index for semantic interest-to-course and course-to-career matching.
Structured transactional data	`enrollments` , `prerequisites_graph` , `graduation_requirements` , `course_success_rates`	Queried read-only via MCP for eligibility checks, prerequisite validation, and GPA-path feasibility.

5. CockroachDB tools — 3 of 4

(Agent Skills Repo was the 4th option, not used — see cost/time tradeoff discussion in the build log.)

Tool	What it does here
MCP Server	Live, read-only structured queries against enrollment, prerequisite, and requirement tables. Full audit logging, no custom proxy.
Distributed Vector Indexing	Semantic matching between a student's free-text goal and course/career embeddings, without a separate vector store.
ccloud CLI	Flask shells out to `ccloud cluster info <name> -o json` and `ccloud audit list --limit 5 -o json`; the agent reports cluster health/audit status on request, demonstrating the memory layer is monitored, not just used.

6. AWS services

- **Amazon S3** — stores generated recommendation reports and CSV dataset backups via `boto3`. Minimal cost; see estimate below.

7. Request lifecycle

1. Student submits a goal → Flask writes a new row to `recommendation_sessions` (task-state write #1) and appends to `conversations`.
2. Flask calls the Claude agent with three tools registered: MCP query, vector search, ccloud health check.
3. Agent calls MCP to check eligibility/prerequisites (memory read).
4. Agent calls the vector tool to semantically match the goal to courses/careers (memory read).
5. Optionally, agent calls ccloud CLI to confirm cluster health before returning results.
6. Agent returns ranked recommendations with short explanations.
7. Flask updates `recommendation_sessions` to `complete` (task-state write #2), appends the agent's reply to `conversations`, and writes a copy of the report to S3.
8. Flask renders results to the student.

8. Deployment

- **Code**: GitHub, public repo, MIT or Apache 2.0 license visible in the About section.
- **Hosting**: Render.com, free web service tier (512 MB RAM, 750 instance-hours/month). Auto-deploys on push to `main`.
- ⚠️ **Flag**: the hackathon brief's intro line says the app should be *"deployed on AWS,"* separate from the *"use ≥1 AWS service"* requirement list. Render satisfies the latter (via S3) but not the former literally. Cheapest fix if you want to close this gap without much extra work: swap Render for **AWS App Runner** (deploys directly from GitHub, comparable simplicity, still mostly free-tier at hackathon scale) or **Elastic Beanstalk**. Otherwise, document clearly in the README that S3 is your AWS integration and let judges interpret "deployed on AWS" as "uses AWS infrastructure."

9. Tech stack

Layer	Choice
Agent	Claude API (tool use)
Backend	Flask (Python)
Database	CockroachDB Cloud (Basic/serverless)
Embeddings	sentence-transformers, `all-MiniLM-L6-v2`, run locally (free)
Object storage	Amazon S3
Hosting	Render.com (see flag above)
Source control	GitHub

10. Estimated cost

Item	Cost
CockroachDB Cloud (Basic)	Free — 50M RUs + 10 GiB/month included
Claude API	~$1–10 total (Haiku for dev/test, Sonnet 5 for final demo)
Amazon S3	Cents — well under free-tier/credit allowances
Render.com	Free tier
Total	**Under $10 for the whole hackathon**

11. Requirements checklist

- [x] Agentic app with CockroachDB as persistent memory layer
- [x] Stores/retrieves/acts on conversation history, user context, task state, embeddings, structured data
- [x] ≥2 CockroachDB tools used → **3 used** (MCP Server, Vector Indexing, ccloud CLI)
- [x] ≥1 AWS service used → S3
- [ ] "Deployed on AWS" (literal reading) — flagged above, decide before submission
- [ ] Public repo + license
- [ ] Functional demo app URL
- [ ] Demo video (<3 min) showing the memory layer at work
- [ ] Architecture diagram — this document